06 堆積排序

Part II Sorting and Order Statistics

本章定位

Binary heap以陣列索引表示近似完全二元樹，支援取極值與優先權更新。本章把父子索引、heap-size及每輪交換對齊，再證明由下往上建堆的線性成本，並區分排序與priority queue的操作需求。

前置知識

完全二元樹、陣列索引與排序不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以 max-heap 性質操作最大優先權佇列。
- 證明 max-heapify 成本與 O(n) 線性建堆。
- 用堆積完成選擇排序式流程並分析 O(n log n)。
- 6.1 Heaps
- 6.2 Maintaining the heap property
- 6.3 Building a heap
- 6.4 The heapsort algorithm
- 6.5 Priority queues

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps

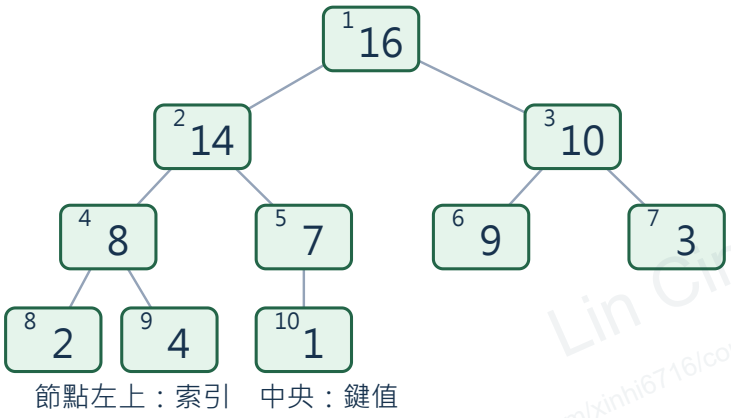
Binary heap的形狀是近似完全二元樹。除最後一層外皆填滿，最後一層由左向右填。陣列採1-index，根1，parent(i)=⌊i/2⌋，left(i)=2i，right(i)=2i+1；只有索引≤heap-size的節點存在。

max-heap的條件是每個非根i滿足A[parent(i)]≥A[i]。沿祖先鏈反覆比較即知根是最大鍵，但同層、左右兩棵子樹沒有排序關係，因此不可直接binary search任意鍵。

陣列length與heap-size不同。heapsort把固定好的尾端排除在heap-size之外，不能讓heapify再碰尾端。若有n≥1個heap節點，深度h的最左索引為2^h，所以樹高⌊log2 n⌋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.1 Heaps (續)



索引固定，鍵值移動

1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

每條邊都滿足：父鍵 ≥ 子鍵
同一列的節點不保證有序

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.2 Maintaining the heap property

MAX-HEAPIFY(A,i,s)的前提是左右child子樹已各自為max-heap，只有i處可能小於child；s是heap-size。若i不在1..s，應由介面拒絕或只對空heap略過呼叫。

```
while true:
    largest=i
    if 2i<=s and A[2i]>A[largest]: largest=2i
    if 2i+1<=s and A[2i+1]>A[largest]: largest=2i+1
    if largest==i: return
    swap(A[i],A[largest])
    i=largest
```

先在兩child中取較大再與父比較，也與此偽碼等價。只因一個child比另一個child大就交換、完全不比較父，則不正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Heapify 的正確性與高度成本

若三者最大就是父，兩條父子邊皆合法，子樹原本合法，所以整棵完成。否則最大child的根鍵也不小於其所有後代，把它換到父後，父到兩child的邊皆合法；唯一可能違規的是舊父鍵下移後的新位置。

以剩餘高度歸納，下移的child子樹會被正確修復，另一子樹不變。每次i變為2i或2i+1，嚴格下降一層，故終止。高度h的子樹最壞O(h+1)，根最壞O(log(n+1))；若父原已大於child，實際可O(1)。

例A=[16,4,10,8,7,9,3]、s=7，i=2比較8與7，交換4↔8，得[16,8,10,4,7,9,3]。到i=4時child索引8、9都大於7，立刻停止；值3在索引7，是節點3的child，不是節點4的child。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

6.3 Building a heap

```
s=length(A)
for i=floor(s/2) down to1:
    MAX-HEAPIFY(A,i,s)
```

迴圈開始時，每個索引大於i的節點都是合法heap的根。初始化時，⌊n/2⌋+1..n都是葉，條件成立。處理i時其child索引較大，兩棵子樹可套heapify前提；完成後i也成為合法heap根，維持下一輪。i降至0時根1合法，整個陣列完成。

空或單元素陣列不執行迴圈，已是heap。i=⌊n/2⌋不一定是葉：n=10時i=5仍有child10，必須比較A5與A10。

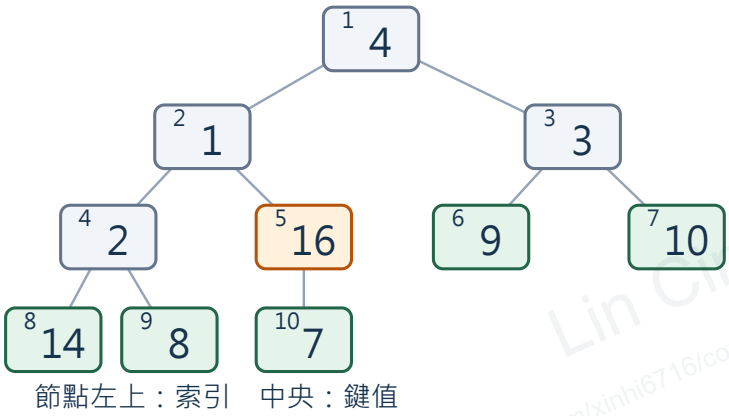
五輪完整陣列與交換

輸入A=[4,1,3,2,16,9,10,14,8,7]，s=10。

i	比較與交換順序	本輪完成的完整陣列
5	A5=16≥A10=7，無交換	[4,1,3,2,16,9,10,14,8,7]
4	child鍵14、8選14；索引4↔8	[4,1,3,14,16,9,10,2,8,7]
3	child鍵9、10選10；索引3↔7	[4,1,10,14,16,9,3,2,8,7]
2	14、16選16；2↔5，再5↔10	[4,16,10,14,7,9,3,2,8,1]
1	先選16再14再8；1↔2、2↔4、4↔9	[16,14,10,8,7,9,3,2,4,1]

以下五張圖逐輪對上同一份資料。橘框是本輪處理的根，綠框標示已保證是heap的子樹根；灰框仍待處理。索引不隨交換改變，改變的是該位置的鍵值。

建堆i=5：與唯一Child比較



節點左上：索引　中央：鍵值

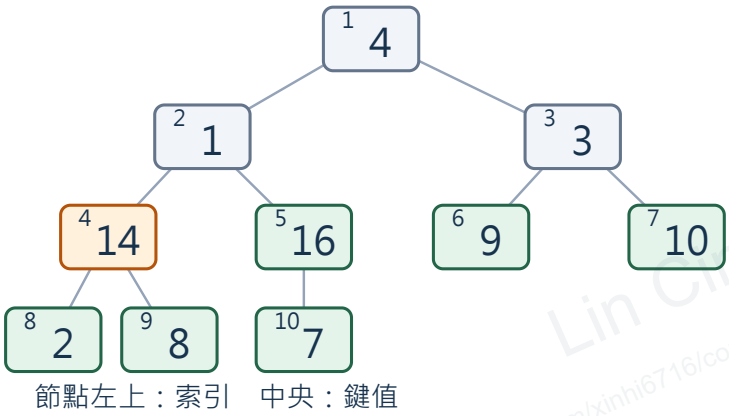
索引固定，鍵值移動

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置5的16已大於位置10的7，沒有交換。位置6..10是葉；位置5的整棵子樹現在也已合法。

建堆i=4：把14提升



節點左上：索引　中央：鍵值

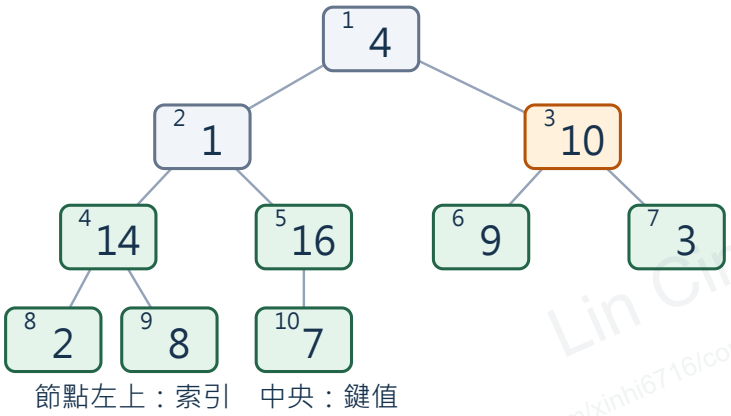
索引固定，鍵值移動

1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置4值2，children分別14、8。先選最大14，交換索引4、8，舊2到葉8停止。沒有把索引9的8也換走。

建堆i=3：把10提升



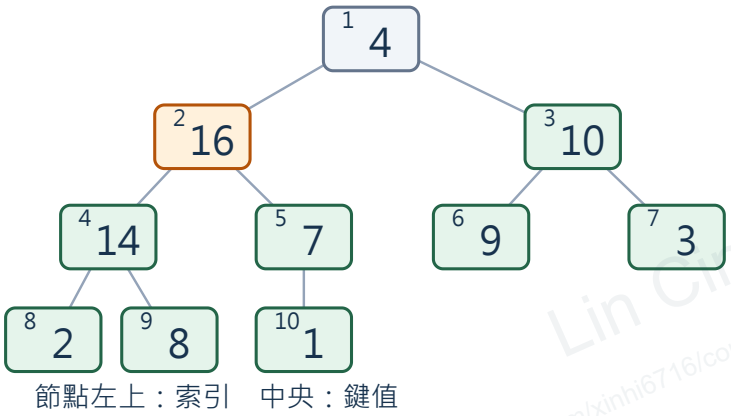
索引固定，鍵值移動

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置3值3，children是索引6的9與索引7的10。交換3、7，舊3到葉7停止；索引6的9不變。

建堆i=2：沿5再到10



索引固定，鍵值移動

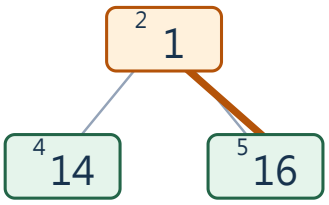
1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

原位置2值1，先與索引5的16交換；舊1到了位置5，還有child10的7，再交換5、10。索引4的14並沒有因本輪被下移。

i=2：舊鍵 1 如何下沉？

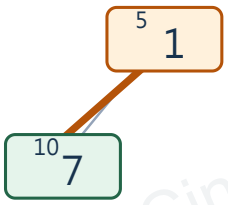
比較 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

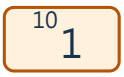
比較 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

到葉，停止



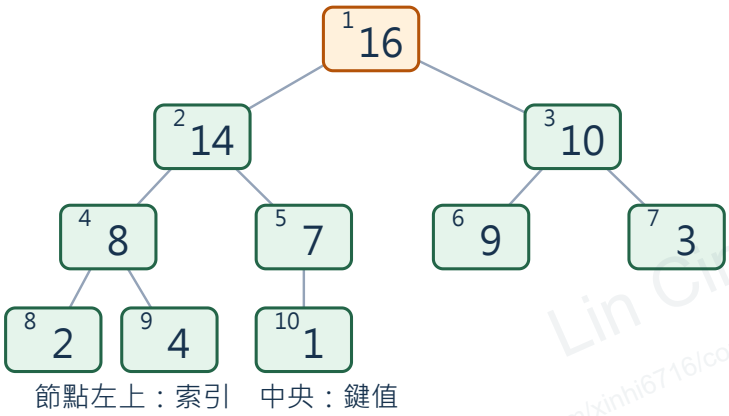
20, 21 > heap-size 10

不再讀取 child

舊鍵 1 留在索引 10

橘框追蹤同一個舊鍵 1；粗邊指出這次選中的較大 child。每個節點左上是索引，中央是鍵值；未畫出的子樹保持不變。

建堆i=1：沿2、4、9下降



索引固定，鍵值移動

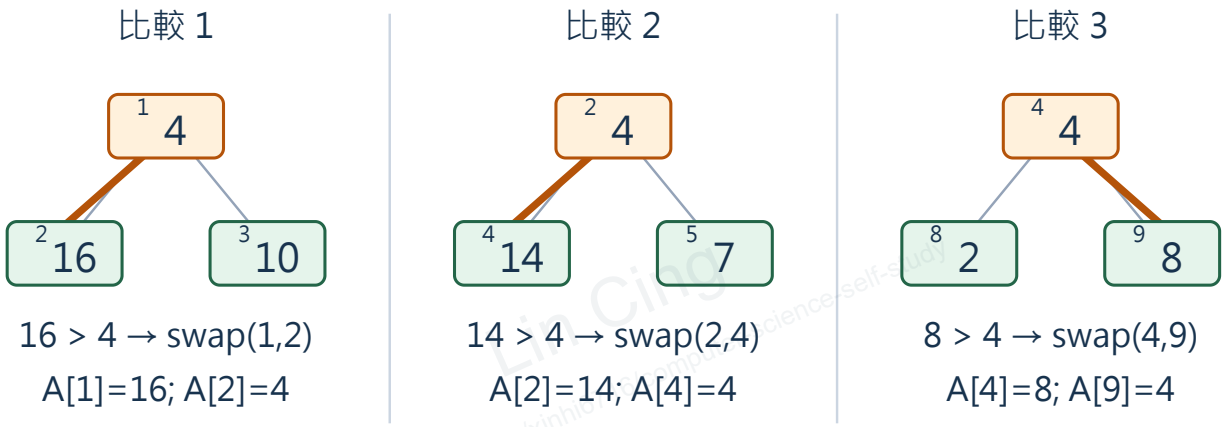
1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

原根4依次與16、14、8交換，停在索引9。最後父子逐一為16≥14,10；14≥8,7；10≥9,3；8≥2,4；7≥1，十個鍵無增減。

i=1：舊根 4 的三次比較



每框都是交換前的局部狀態，底行列出交換後的兩個鍵。4 到索引 9 後是葉；索引 18、19 超過 heap-size=10，停止。不是沿最終陣列的數值順序走訪。

每次交換後，都能核對完整陣列

固定索引為 1..10；下表接續 i=3 完成後的陣列。與前兩張局部圖逐步對照。

本輪根	交換索引	交換後 A[1..10]
2	2 ↔ 5	[4,16,10,14,1,9,3,2,8,7]
2	5 ↔ 10	[4,16,10,14,7,9,3,2,8,1]
1	1 ↔ 2	[16,4,10,14,7,9,3,2,8,1]
1	2 ↔ 4	[16,14,10,4,7,9,3,2,8,1]
1	4 ↔ 9	[16,14,10,8,7,9,3,2,4,1]

自我檢查：遮住結果，先選較大的 child，再判斷是否比父大。每次只能交換兩格；其他八格與元素多重集合不變。

為何Build-Heap是線性

令height(i)為位置i子樹高度。它至少h高，當且僅當最左的h代後裔索引i×2^h≤n，所以至少h高的節點數恰為⌊n/2^h⌋。

把每節點高度分成一層一層加總，

$$\sum_i \text{height}(i) = \sum_{h \geq 1} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

每次heapify成本O(height(i)+1)，且呼叫數⌊n/2⌋，所以總成本O(n)。對n≥2，迴圈本身要處理⌊n/2⌋個節點，給Ω(n)，故build的時間Θ(n)。例如n=15的高度和為7+3+1=11。

逐項INSERT的上浮高度之和則可Θ(n log n)，那不是同一批由下而上的操作，不能套此高度總和。

6.4 The heapsort algorithm

```
BUILD-MAX-HEAP(A)
for end=length(A) down to2:
    swap(A[1],A[end])
    MAX-HEAPIFY(A,1,end-1)
```

每輪前，A[1..end]為max-heap；A[end+1..n]已遞增排序，且其每個鍵都不小於heap內任一鍵。交換把目前最大值放到A[end]，縮小s後只修剩餘heap，因此不變量維持。end=1時全陣列遞增，且所有操作是交換，元素多重集合不變。

時間O(n log(n+1))，由不同鍵比較排序的資訊下界，最壞時間Θ(n log n)。全相等輸入在本章遇相等即停的heapify實作可能只需Θ(n)，不能把最壞界說成每一份輸入都相同。迭代heapify只用O(1)額外空間；遞迴版另有O(log n)呼叫堆疊。

資訊下界的理由是n個不同鍵有n!種輸入次序，正確比較決策樹至少需要n!個葉；高度d的二元樹最多2^d葉，所以d≥log2(n!)。乘積後半至少⌊n/2⌋項各≥n/2，故log2(n!)≥⌊n/2⌋log2(n/2)=Ω(n log n)。因此至少一種輸入要這麼多次比較。

十元素Heapsort的全部輪次

每輪先swap根與heap末端，再heapify剩餘前綴。

end後的新s	完成修復的heap	已排序尾端
9	[14,8,10,4,7,9,3,2,1]	[16]
8	[10,8,9,4,7,1,3,2]	[14,16]
7	[9,8,3,4,7,1,2]	[10,14,16]
6	[8,7,3,4,2,1]	[9,10,14,16]
5	[7,4,3,1,2]	[8,9,10,14,16]
4	[4,2,3,1]	[7,8,9,10,14,16]
3	[3,2,1]	[4,7,8,9,10,14,16]
2	[2,1]	[3,4,7,8,9,10,14,16]
1	[1]	[2,3,4,7,8,9,10,14,16]

十元素Heapsort的全部輪次 （續）

第二輪原根1只沿索引1→3→6下沉，與10、9交換；沒有再與8或2交換。追蹤的是交換路徑，不可把最終陣列中的所有值串成路徑。

6.5 Priority queues

max-priority queue的MAXIMUM直接讀A1，O(1)；EXTRACT-MAX先保存A1、以最後鍵補根、s減1，再heapify，最壞O(log(n+1))。空時須回傳EMPTY，不可讀A1。

INCREASE-KEY(A,i,k)要求k≥原鍵，改鍵後反覆與父比較：若父較小就交換並向上；父已不小於k或到根即停。只有祖先邊可能違規，向上後每次修好一條，故正確且最壞O(log n)。

INSERT在尾端加新位置，先設特殊−∞再increase-key；也可直接寫新鍵後執行同一上浮，不需型別真的能表示−∞。動態陣列擴容的成本須另作攤銷分析；固定容量版本先檢查FULL。

例A=[20,15,18,7,12,9,17,2]，位置8升為16。先8↔4得[20,15,18,16,12,9,17,7]，再4↔2得[20,16,18,15,12,9,17,7]；到父20停止。

降鍵、指定刪除與穩定性

max-heap降鍵只可能破壞往child的邊，因此向下heapify；升鍵只向上。刪指定位置i可用末鍵補i、s減1，若新鍵大於父就上浮，否則向下heapify。若i就是末端，直接縮短即可。兩分支皆最壞O(log n)，但搜尋任意鍵找到i仍可能Θ(n)。

若外界用handle指定元素，每次swap都要同步更新handle到陣列位置的對照；陣列索引本身不會永久指向同一筆資料。

heapsort一般不穩定。例如(2,a),(2,b),(1,c)，相等鍵heapify不交換時，最終可成(1,c),(2,b),(2,a)，同鍵a、b順序反轉。需要穩定性時可另保存原序號作比較鍵，但這會改變比較規則與儲存模型。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 06。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study